

GenAI Engineer — Take-Home Assignment

Building an Agentic Document Intelligence Assistant

Objective

Build a system that lets a user upload one or more documents (PDF, DOCX, or scanned images) and interact with them conversationally. The assistant should answer questions, summarize content, and extract structured data — all grounded in the uploaded documents — using a Retrieval-Augmented Generation (RAG) pipeline with agentic (tool-using) capabilities.

Problem Context

Teams routinely need to pull answers, summaries, and structured data out of long or messy documents (contracts, reports, manuals, forms). A good GenAI system should not just paraphrase text — it should retrieve the right context, reason over it, decide when to call tools or ask for clarification, and produce grounded, verifiable output rather than hallucinated answers.

Core Requirements

1. Document Ingestion

- Accept PDF and/or DOCX upload (image/OCR input is a bonus, not required).
- Parse and chunk documents in a way that preserves useful context (e.g., section-aware or semantic chunking, not arbitrary fixed-size splitting).

2. Retrieval-Augmented Generation (RAG) Pipeline

- Generate embeddings and store them in a vector index of your choice (e.g., Chroma, Weaviate, pgvector, Pinecone).
- Implement retrieval that returns relevant, ranked chunks for a given query.
- Justify your chunking strategy, embedding model choice, and retrieval parameters (top-k, similarity threshold, etc.) in the README.

3. LLM Integration & Prompt Design (Mandatory Constraint)

- Use an LLM to synthesize answers from retrieved context — API-based (OpenAI/Anthropic/etc.) or self-hosted (Ollama/llama.cpp) is your choice, but explain the trade-off you made.
- Design prompts/system instructions that keep the model grounded in retrieved context and explicitly avoid answering when the context doesn't support an answer.
- Responses should cite or reference which chunk(s)/section(s) of the source document(s) they're grounded in.

4. Conversational Interface

- Support multi-turn conversation with memory of prior turns (not just single-shot Q&A).
- Handle follow-up questions and pronoun/context references correctly (e.g., "what about the second clause?").

5. Agentic Capabilities (Key Requirement)

The assistant should behave as an agent, not just a Q&A wrapper. At minimum, implement tool use for:

- Structured extraction — e.g., "pull out all dates and amounts into a table" should trigger a tool call that returns validated JSON, not free text.
- At least one additional tool of your choice (e.g., a calculator, a document comparison tool, a web/API lookup, or a summarization tool invoked conditionally rather than always).
- The agent should decide when a tool is needed versus when to answer directly from retrieved context — this decision logic should be visible/explainable in your write-up.

6. Output Structuring & Validation

- Structured outputs (e.g., extracted fields) should conform to a defined JSON schema.
- Validate LLM output against the schema and handle malformed/hallucinated output gracefully (retry, repair, or flag).

7. Architecture Expectations

Design for:

- Clear separation of concerns — ingestion, retrieval, LLM/agent orchestration, and interface layers.
- Conversation/session state handling across turns.
- Reusability — the pipeline should work on a new document without code changes.

Bonus Considerations

- Streaming responses (token-by-token).
- Guardrails or hallucination detection layer.
- Multi-document / cross-document reasoning.
- Support for a self-hosted or fine-tuned local LLM as an alternative backend.
- Cost/latency optimization (caching, batching, smaller models for routing decisions).

Deliverables

- GitHub repository with source code.
- Architecture diagram.
- README covering:
 - LLM/embedding model choices and why
 - Prompt design and grounding strategy
 - Agent/tool-use design and decision logic
 - Known limitations and what you'd improve with more time
- Demo video, or in-person demo and walkthrough.

Evaluation Criteria

- **Retrieval quality** — relevance and precision of retrieved context.
- **Grounding & hallucination control** — does the system avoid answering beyond what the documents support?
- **Prompt engineering & LLM orchestration** — clarity, robustness, and reasoning quality of prompts.
- **Agentic reasoning** — sound tool-use decisions and correct execution.

- **Structured output correctness** — schema validity of extracted/generated JSON.
- **Code quality & architecture** — separation of concerns, readability, extensibility.
- **UX quality** — usability of the conversational interface.

Suggested References

- [LangChain documentation](#)
- Vector databases: Chroma, Weaviate, pgvector, Pinecone
- [Ollama](#) / [llama.cpp](#) for self-hosted LLMs